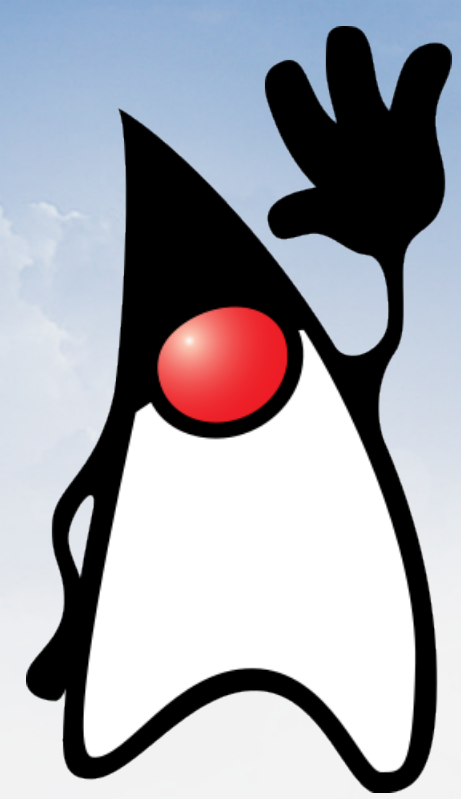




Collections in Java

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Collections

- Java's standard API has common data structures that we can use for different purposes. We call those "collections".
- Collections work with generics. We specify the data type in angle brackets. <Object>
- LinkedList, ArrayList, HashSet, HashMap, TreeMap are examples.
- There is also a static class called "Collections" under java.util.Collections.

Collections class

```
Collections.sort(new ArrayList<Integer>());  
// sorts the List, modifies the list.  
Collections.shuffle(new ArrayList<Integer>());  
// shuffles the List, modifies the list.
```

Queue

boolean isEmpty() : Tests if this queue is empty.

E peek(): returns the top of this queue without removing

E poll(): removes the head and returns the object, if isEmpty() returns null

E add(E item): pushes an item onto the tail of this queue.

```
import java.util.Queue;
import java.util.LinkedList;

public class QueueTester {
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>();
        queue.add("one");
        queue.add("two");
        System.out.println(queue);
        System.out.println(queue.poll());
        System.out.println(queue.poll());
        System.out.println(queue.poll());
    } //end of main
}
```

```
[one, two]
one
two
null
```

Queue is an interface, we are in fact using methods of LinkedList<>

Stack

boolean empty() : Tests if this stack is empty.

E peek(): returns the top of this stack without removing

E pop(): removes the top and returns the object

E push(E item): pushes an item onto the top of this stack.

int search(Object o): Returns the distance from the top of the searched element

```
package stackTester;
import java.util.Stack;

public class StackTester {
    public static void main(String[] args){
        Stack<Integer> arr =new Stack<>();
        arr.push(3)
        arr.push(7);
        arr.push(1);
        arr.push(3);
        System.out.println(arr.search(1));
    }
}
```

2

the second pop will return this element.



list and set



LIST

A list is like a line of people or items — they have a specific order, and duplicates are allowed. Imagine students lining up for ice cream.

- Some students might be twins (same face, same clothes — duplicates!), and that's okay.
- The order in line matters: who is first, second, third, etc.

Key points: Order (insertion order is preserved) matters, duplicates allowed.

SET

A set is like a collection of unique things. It doesn't allow duplicates. All of the keys should be unique. Order doesn't matter — what matters is what's inside.

Imagine you are collecting stickers.

- You only want one of each sticker. When you get one, you stick to your notepad somewhere.
- If someone gives you the same sticker again, you just smile and keep your original one.
- It doesn't matter where you put them in your book — just that you have them.
- When someone asks you to see all of them, you show the book but which one is given first or last doesn't matter. Santa always finds the toy from unorganized sack immediately.

Key points: No duplicates, no special order/organization

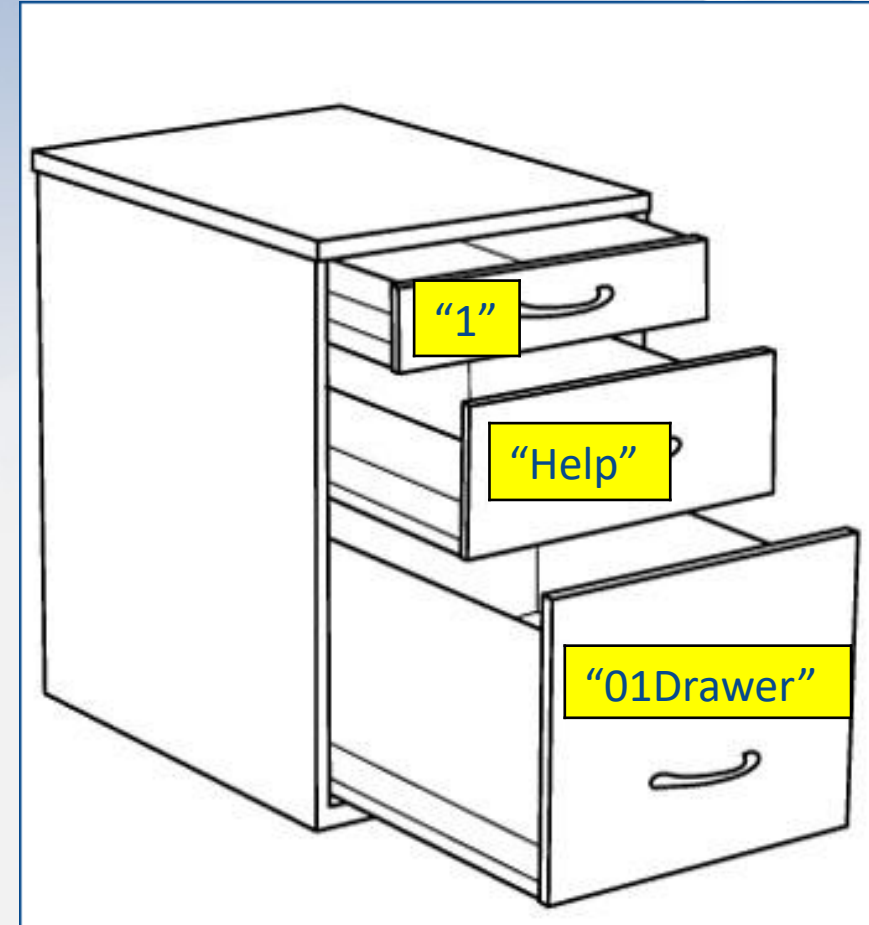
HashMap/dictionary

- A hashmap is like a box of labeled drawers.
- Each drawer (key) has something inside it (value).
- You find things quickly by reading the label instead of opening every drawer.

Think about a school locker.

- Each locker has a post-it where a name or number is written(key).
- Inside, there are books, snacks, whatever (value).
- If you know the locker number, you can directly open it and find what's inside
 - no need to check all lockers!

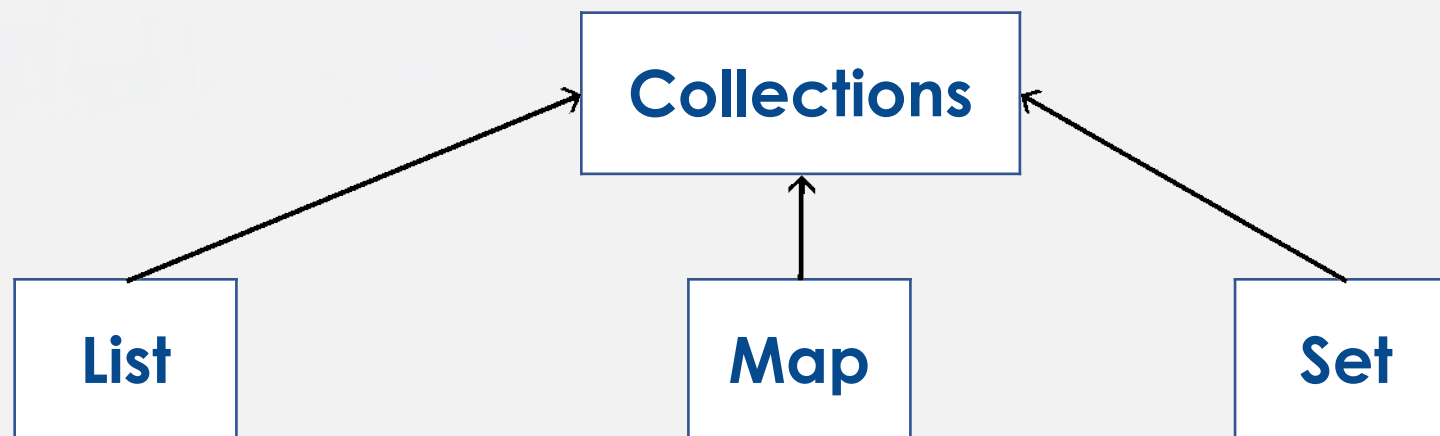
Key points: Key → Value. Quick access using the key.



Set can be also thought as drawers where the items inside doesn't matter. If all items inside the drawers are all same (ex: 1, true), then it means you have a set.

Collections

Lists	Sets	Map
LinkedList, ArrayList<Key>	HashSet<Key>	HashMap<Key,Value>
Numeric indexed	Non numeric indexed	Non numeric indexed
Duplicates are OK	No Duplicate keys	No Duplicate keys
Insertion order is preserved.	Insertion order is NOT preserved.	Insertion order is NOT preserved.
Multiple null values are allowed.	Only one null key is allowed.	Only one null key is allowed.



In HashMap and HashSet, the keys are unique.

LinkedList and ArrayList Classes

- Both classes have indexed based methods.
 - add(int index, E element)
 - get(int index)
 - indexOf(E element)
 - remove(int index)
 - remove(E element)
 - contains(E element)
- Both classes allow us to specify order of elements
- Implements the *List* Interface

ArrayList and LinkedList

Method

Description

add(E e)

Inserts element at the end of the list

add(int index, E e)

Inserts element at the specified position

clear()

Makes the list empty

get(int index)

Retrieves index at specified index

remove(Object o)

Remove element from list (first occurrence)

remove(int index)

Remove element from specified index of the list

size()

Number of elements in the list

```
import java.util.*; // List-> java.util.List;
```

```
public class Tester1 {  
    public static List<Integer> getFilledList() {  
        List<Integer> arr = new ArrayList<Integer>();  
        arr.add(null);  
        arr.add(null);  
        for(Integer el:arr){  
            if(el!=null){  
                System.out.println(el.toString());  
            } else{  
                System.out.println("NULL");  
            }  
        }  
        return arr;  
    }  
    public static void main(String[] args) {  
        ArrayList<Integer> arr = (ArrayList)getFilledList();  
        System.out.println(arr);  
    }  
}
```

```
NULL  
NULL  
[null, null]
```

Collections

Only classes which extends *List* are sortable and shufflable. Not Maps or Sets

`Collections.sort(CollectionObj)`

Performs an ascending rearrangement of elements

`Collections.shuffle(CollectionObj)`

Performs a randomize rearrangement of elements

Important! If we define our own user defined class, we must implement the Comparable Interface to override the compareTo method for proper sorting.

```
import java.util.*; // Collections are inside java.util.Collections
public class CollectionTester{
    public static void main(String[] args) {
        LinkedList<Integer> arr=new LinkedList<Integer>();
        arr.add(3);arr.add(1);arr.add(84);
        arr.add(2);arr.add(44);arr.add(-2);//arr.add(null);
        System.out.println(arr);
        Collections.sort(arr);
        System.out.println(arr);
        Collections.shuffle(arr);
        System.out.println(arr);
    } //end of main method
}
```

```
[3, 1, 84, 2, 44, -2]
[-2, 1, 2, 3, 44, 84]
[44, 1, 2, -2, 84, 3]
```

```
[3, 1, 84, 2, 44, -2, null]
```

ERROR!

Exception in thread "main"

java.lang.NullPointerException: Cannot invoke "java.lang.Comparable.compareTo(Object)" because "pivot" is null

HashSet

- Hash tables can be widely used to implement a lot of data structures. The Java Collections library has a HashSet Class that implements a set based hash table.
- A set is a collection of elements with no duplicates.
- All elements are unique. **We call each element as key.**
- Only one null key is allowed.
- Implements the Set Interface.
- Set data structure is created to add/search elements fast, in $O(1)$ time.

HashSet

Methods	Description
boolean add(E e)	Adds the specified element to this set if it is not already present. If present nothing is done.
void clear()	Removes all of the elements
boolean contains(Object o)	Returns true if this set contains the specified element
boolean isEmpty()	Returns true if this set contains no elements
boolean remove(Object o)	Removes the element. If it exists returns true
int size()	Returns the number of elements

HashSet looks like an array where there are no positions as 0,1,2,3 .. but String,Integer keys. Those keys are unique in the set. Can be thought as a drawers where each drawer is marked with a note paper

"Java"	"0"	...	"55"	...
--------	-----	-----	------	-----

HashSet

```
package hashmapTester;
import java.util.*;

public class Tester1 {
    public static void main(String[] args) {
        HashSet<Integer> arr=new HashSet<>();
        arr.add(4);arr.add(99);arr.add(5);arr.add(-1);
        System.out.println( "Have we added -1? " + arr.add(-1) );
        for(Integer e:arr){
            if(e!=-1){
                System.out.print(e.toString()+" ");
            }else{
                arr.remove(Integer.valueOf(e));
            }//end of if
        }//end of for
        System.out.println(arr);
    }//end of main method
}//end of class
```

Have we added -1? false
ERROR!
Exception in thread "main"
java.util.ConcurrentModification
Exception at java. base/java.util.
HashMap\$HashIterator.nextNode
(HashMap.java:1511)
at java.base/java.util.
HashMap\$KeyIterator.next(Hash
Map.java:1534) at
Tester1.main(Tester1.java :10)

Shorthand version of
traversal does not allow
removing

HashMap

- Hashmap keeps the values with keys
 - Entry<Key, Value>
- Keys will be unique, no duplicates allowed
- Insertion order will not be preserved.
- HashMap in some point of view, stores (key,value) pairs. When we traverse, we will traverse list of those pairs.
- Searching/Accessing the key and then value is very fast. $O(1)$.
- Nice structure for storing frequencies of a list with only one traversal.
 - When you traverse a list you can store each element as key inside a hashmap. If you have seen that number before, just increase the value for that key/element. (Can you implement this?)

Why do we need HashMap?

1, 45, 2, 99, 28, 53

- You can either store them in list but you need linear search($O(n)$) OR binary search($O(\log n)$)

- You create an array with max size as maximum element
 $\text{arr}[45] \Rightarrow 45, \text{arr}[1] \Rightarrow 1, \text{arr}[99] \Rightarrow 99$

With that solution you will find the element in $O(1)$ but you are wasting 93 spaces for 6 items.

0	
1	1
2	2
3	53
4	
5	45
6	
7	
8	28
9	99

Store the numbers inside this small table. Use remainder operation to decide the position of the element. finding remainder for any number is easy

Wasting 4 spaces only.

1 is at $1\%10 \Rightarrow 1$

28 is at $28\%10 \Rightarrow 8$

What if we need to add 75 to the hashmap again? That would be collision and there are ways to solve that. Ex: create a bigger table with a different size.

HashMap

Method	Description
put(Key,Value)	Associates a value with the specified key
containsValue(Value)	Returns true if the value exists
containsKey(Key)	Returns true if the key exists
get(Key)	Returns the value. null if no key found.
size()	Returns the size of the hashmap as int
entrySet()	Returns a set of pairs, <key,value>. For traversal, this set will be traversed.
remove(key)	removes the key if present

HashMap

```
package hashmapTester;
import java.util.HashMap;
import java.util.Map;

public class HashMapTester {
    public static void main(String[] args) {
        // Key , Value
        HashMap<String,Integer> hm = new HashMap<>();
        hm.put("Java", 34);
        hm.put("Javascript", 12);
        hm.put("C++", 45);
        // Traversing a list ( entry set) of tuples
        for (Map.Entry<String,Integer> mapElement : hm.entrySet()) {
            String key = mapElement.getKey();
            int value = mapElement.getValue();
            System.out.println(key + " : " + value);
            // hm.remove("Java"); not possible
        }
        if(hm.containsKey("Java")){
            System.out.println("Java has "+hm.get("Java")+" inside");
        }
    } //end of main method
}
```

Java : 34

C++ : 45

Javascript : 12

Java has 34 inside

Removal is not possible while traversing the HashMap in this manner. We will get a runtime error.

java.util.ConcurrentModificationException

TreeMap and TreeSet

- `TreeSet<Key>` and `TreeMap<Key,Value>`
- `HashSet` and `HashMap` doesn't allow duplicates. `TreeMap` and `TreeSet` doesn't allow duplicates also.
- `Tree*` is sorted whereas `Hash*` is not sorted.
- The same methods exists for
 - `HashMap,HashSet => TreeMap,TreeSet`
- Run time complexity of `Tree*` operations are $O(\log n)$.

